

UNIVERSIDAD DE CALDAS

Facultad de Inteligencia Artificial e Ingenierías
Departamento de Ingeniería de Sistemas y Computación

TacticalGrid

Manual de Usuario

Sistemas Inteligentes I

Autores
Jorge Iván Garcia Torres
Daniel Felipe Franco Rincón

Docente
Jairo Iván Vélez Bedoya

Materia
Sistemas Inteligentes I

Entrega
Octubre de 2026

https://github.com/df1903/tactical-grid-ai

Manizales, Caldas – Colombia
2026

Índice general

1. Requisitos e Instalación **1**
- 1.1. Requisitos Previos . . . 1
- 1.2. Instalación Paso a Paso . . . 1
- 1.3. Verificación Rápida . . . 2

2. Inicio Rápido **3**
- 2.1. Ejecutar la Interfaz Gráfica (GUI) . . . 3
- 2.2. Ejecutar una Búsqueda (CLI) . . . 3
- 2.3. Jugar una Partida (GUI) . . . 4
- 2.4. Jugar una Partida desde la Terminal . . . 4
- 2.5. Atajos de Teclado / Mouse . . . 4

3. Interfaz Gráfica (GUI) **5**
- 3.1. Descripción General . . . 5
- 3.2. El Mapa (Canvas Central) . . . 8
- 3.3. Panel Derecho (Controles) . . . 8
- 3.4. Interacción: Flujo Típico . . . 9
- 3.5. Leyenda (Debajo del Mapa) . . . 9

4. Línea de Comandos (CLI) **11**
- 4.1. Uso Básico . . . 11
- 4.2. Opciones Disponibles . . . 12
- 4.3. Listas Comma-Separated (Sweep Combinatorio) . . . 12
- 4.4. Variante dinámica . . . 13
- 4.5. Ejemplos de Uso Común . . . 13
  - 4.5.1. Búsqueda simple con verificación . . . 13
  - 4.5.2. Adversarial con verificación (check 6) . . . 14
  - 4.5.3. Partida completa desde la terminal . . . 14
  - 4.5.4. Sweep de rendimiento (caso 10.5 del informe) . . . 14
  - 4.5.5. Sweep completo para el informe (todos los casos) . . . 14
- 4.6. Salida JSON (Estructura) . . . 15
  - 4.6.1. Modo `busqueda` . . . 15
  - 4.6.2. Modo `adversarial` . . . 15
  - 4.6.3. Modo `partida` . . . 16

5. Formato de Archivo de Escenario (JSON) 17
5.1. Estructura Básica . . . 17
5.2. Campos Obligatorios . . . 19
5.3. Campos Opcionales del Equipo (RNF-02) . . . 20
5.4. Variante dinámica . . . 20
5.5. Reglas de Validación (RF-34, RF-35) . . . 20
5.6. Escenarios Incluidos . . . 21
5.7. Crear un Escenario Nuevo . . . 21

6. Reproducir los Experimentos Reportados 22
6.1. Preparación . . . 22
6.2. Casos 1–2: Búsqueda No Informada y Costos (F2) . . . 22
6.3. Casos 3–5: Búsqueda Informada y Beam (F3) . . . 23
6.4. Casos 6–10: Adversarial (F4) . . . 24
6.5. Rendimiento (G13) . . . 25
6.6. Sweep Completo (Todos los Casos 1-10 + Apéndice A) . . . 26
6.7. Caso 11: Variante dinámica . . . 26
6.8. Verificación de que Todo Pasa . . . 26

7. Tests, Lint y Calidad 28
7.1. Ejecutar Tests . . . 28
7.2. Descripción de Suites de Test . . . 29
7.3. Calidad de Código . . . 30
7.4. Configuración de Ruff (pyproject.toml) . . . 30
7.5. Configuración de MyPy (pyproject.toml) . . . 30
7.6. Docstrings y Validación Estudiantil . . . 31

8. Solución de Problemas Comunes 32
8.1. Problemas de Instalación y Ejecución . . . 32
8.2. Errores de Escenario (Exit Code 2) . . . 32
8.3. Problemas de Algoritmos y Resultados . . . 33
8.4. Rendimiento y Memoria . . . 34
8.5. Obtener Ayuda . . . 34
8.6. Declaración de apoyo de IA . . . 35

A. Referencia Rápida de Comandos 36

B. Referencia Rápida de JSON 37

C. Referencia Rápida de Algoritmos 38

D. Diagramas de Referencia 39

Índice de figuras

3.1. Vista principal de la interfaz con el mapa y la zona de controles. 5
3.2. Panel lateral con algoritmo, heurística, profundidad y acciones de ejecución. 6
3.3. Panel de métricas y verificaciones tras ejecutar un algoritmo. 6
3.4. Panel de traza con expansiones, decisiones y acciones de la partida. . . 7
3.5. Modo automático de IA contra IA ejecutándose en la interfaz gráfica. . 7

4.1. Ejecución de la línea de comandos con métricas y verificación de resultados. 11
4.2. Sweep combinado de algoritmos con exportación de resultados en CSV/JSON. 13

5.1. Estructura del escenario JSON cargado y validado por la aplicación. . . 17
5.2. Mensaje de validación del escenario mostrando errores o confirmación correcta. 18

7.1. Salida de pytest, ruff y mypy mostrando la validación de calidad del proyecto. 29

D.1. Arquitectura general del sistema. 39
D.2. Ciclo de turno de la partida. 40
D.3. Flujo de aplicación de acciones. 41

Índice de cuadros

2.1. Atajos en la GUI 4

3.1. Elementos visuales del mapa 8

4.1. Opciones CLI 12

5.1. Campos obligatorios del JSON 20
5.2. Campos opcionales del equipo (no alteran obligatorios) 20
5.3. Escenarios en `scenarios/` 21

6.1. Caso 1: BFS vs DFS 23
6.2. Caso 2: BFS vs UCS 23
6.3. Caso 3: UCS, A* h1, A* h2 23
6.4. Caso 4: UCS, A* h1, A* h2 24
6.5. Caso 5: Beam k=1,2,4,8 24
6.6. Caso 7: Minimax profundidades 1-3 25
6.7. Caso 8: Minimax vs Alfa-beta (prof 4) 25
6.8. Caso 9: Orden default vs reversed (prof 3) 25
6.9. Caso 10: Alfa-beta profundidades 1-5 25

A.1. Resumen de opciones CLI 36

B.1. Campos del formato JSON de escenario 37

C.1. Resumen de algoritmos implementados 38

Capítulo 1

Requisitos e Instalación

1.1 Requisitos Previos

- **Sistema operativo:** Windows 10/11, Linux, macOS
- **uv:** Gestor de paquetes Python que provisiona Python 3.14 automáticamente. Instalación: https://docs.astral.sh/uv/getting-started/installation/

```
# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm
↪  https://astral.sh/uv/install.ps1 | iex"

# Linux/macOS
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- **tkinter:** Viene incluido con Python en Windows. En Linux: `sudo apt install python3-tk`.

1.2 Instalación Paso a Paso

```
# 1. Clonar repositorio
git clone https://github.com/df1903/tactical-grid-ai
cd tactical-grid-ai

# 2. Instalar dependencias (uv crea venv y provisiona Python 3.14)
uv sync

# 3. Verificar instalación (opcional)
uv run pytest                 # 281 tests deben pasar
uv run ruff check .           # lint
uv run ruff format --check .  # formato
uv run mypy src               # tipos strict
```

1.3 Verificación Rápida

```
# Ejecutar la GUI con el mapa de prueba 20x20
uv run python -m tactical_grid.gui scenarios/prueba_20x20.json

# Una búsqueda desde CLI
uv run python -m tactical_grid scenarios/apendice_a.json --algorithm A_STAR
↪  --verify
```

Si la ventana se abre y muestra el grid con colores, unidades, bases, recurso y leyenda, la instalación es correcta.

Capítulo 2

Inicio Rápido

2.1 Ejecutar la Interfaz Gráfica (GUI)

```
# Abrir la GUI con un escenario específico
uv run python -m tactical_grid.gui scenarios/prueba_20x20.json

# Sin argumentos: abre diálogo para elegir archivo
uv run python -m tactical_grid.gui
```

La ventana se abre mostrando:

- **Mapa central:** Grid con terrenos coloreados por costo (verde claro = barato, marrón = caro), obstáculos gris oscuro, bases con borde azul/rojo, recurso (diamante dorado), unidades (círculos azul/rojo).
- **Panel derecho:** Turno actual, movimiento #/límite, portador del recurso, unidad seleccionada, meta de búsqueda y métricas.
- **Traza:** Orden de expansión de búsquedas, valores de las acciones raíz en decisiones y acciones de la partida.
- **Barra inferior:** Botones *Ejecutar*, *Animar*, *Aplicar jugada*, *Reiniciar*, verificación RF-42, selector de bando y *Partida automática*.

2.2 Ejecutar una Búsqueda (CLI)

```
# BFS desde A1 a (3,2) en el ejemplo oficial
uv run python -m tactical_grid scenarios/apendice_a.json \
    --algorithm BFS --unit A1 --goal 3,2 --verify

# A* con heurística obstacle-aware (h2)
uv run python -m tactical_grid scenarios/caso_03_obstaculo_heuristica.json \
    --algorithm A_STAR --heuristic H2 --verify
```

La salida es JSON con claves equivalentes al Apéndice B: `algoritmo`, `exito`, `camino`, `costo`, `longitud`, `estados_generados`, `estados_expandidos`, `maximo_frontera`, `tiempo_segundos`.

2.3 Jugar una Partida (GUI)

1. Abre la GUI: `uv run python -m tactical_grid.gui scenarios/prueba_20x20.json`
2. Modo *Seleccionar / mover unidades* (radio button izquierdo).
3. Clic en una unidad de tu bando (azul = A, rojo = B) → se selecciona (anillo negro discontinuo).
4. Celdas verdes → destinos legales para esa unidad. Clic en una → mueve.
5. Turno pasa automáticamente al otro bando.
6. Gana quien lleva el recurso (diamante dorado) a su base (borde grueso de su color).
7. Empate al llegar al límite de turnos (default 100).

2.4 Jugar una Partida desde la Terminal

```
# IA contra IA hasta terminar la partida
uv run python -m tactical_grid scenarios/caso_07_decision_adversarial.json \
    --mode partida --play auto --algorithm ALPHA_BETA --depth 3

# Persona contra IA; --side B permite jugar segundo
uv run python -m tactical_grid scenarios/caso_07_decision_adversarial.json \
    --mode partida --play humano --side A --depth 3

# Variante dinámica
uv run python -m tactical_grid scenarios/caso_11_dinamico.json \
    --mode partida --play humano --variant dinamica --depth 3
```

En el modo `humano`, la terminal enumera las acciones legales y se selecciona una escribiendo su número; `q` o `salir` abandona la partida.

2.5 Atajos de Teclado / Mouse

Cuadro 2.1: *Atajos en la GUI*

Acción	Tecla o clic
Seleccionar unidad	Clic izquierdo sobre la unidad
Mover unidad	Clic izquierdo en una celda verde (después de seleccionar)
Fijar meta de búsqueda	Clic derecho en cualquier celda
Aplicar decisión	Botón *Aplicar jugada*
Animar búsqueda	Botón *Animar*
Partida automática	Botón *Partida automática*; vuelve a pulsarlo para pausar
Reiniciar partida	Botón *Reiniciar*

Capítulo 3

Interfaz Gráfica (GUI)

3.1 Descripción General

La GUI (`gui/app.py`, `tkinter`) cumple **RF-40**: muestra cuadrícula, terrenos, obstáculos, bases, recurso, unidades, bando, portador, turno y camino encontrado. También permite jugar manualmente contra la IA o dejar que la IA controle ambos bandos.

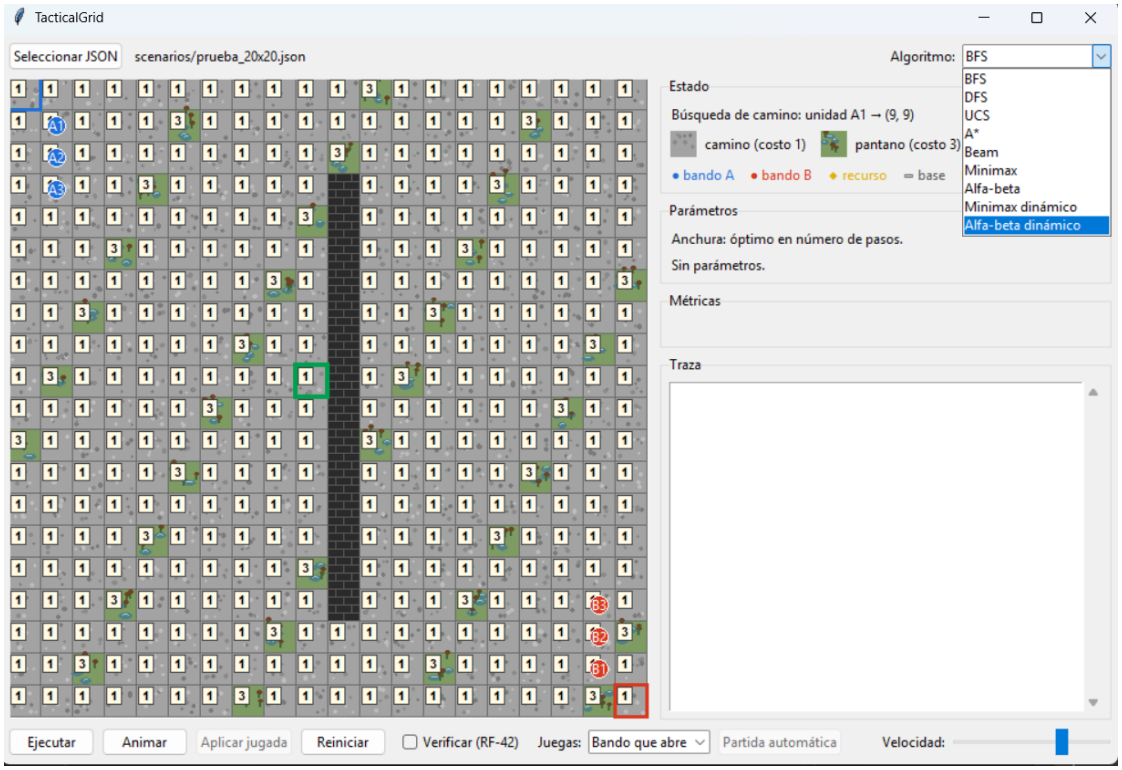


Figura 3.1: *Vista principal de la interfaz con el mapa y la zona de controles.*

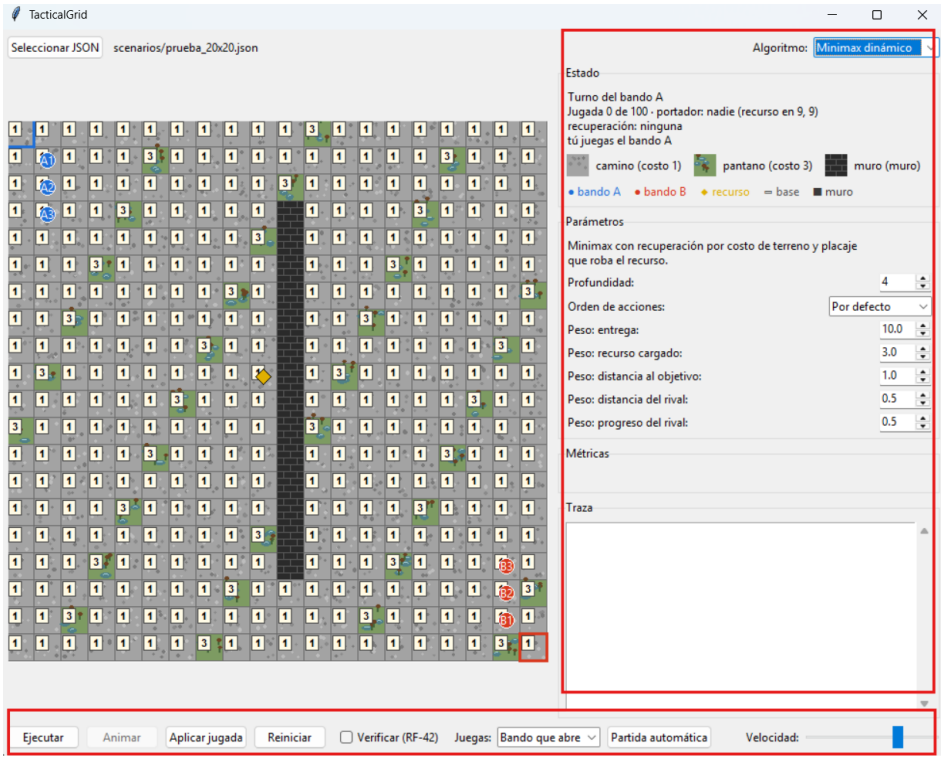


Figura 3.2: *Panel lateral con algoritmo, heurística, profundidad y acciones de ejecución.*

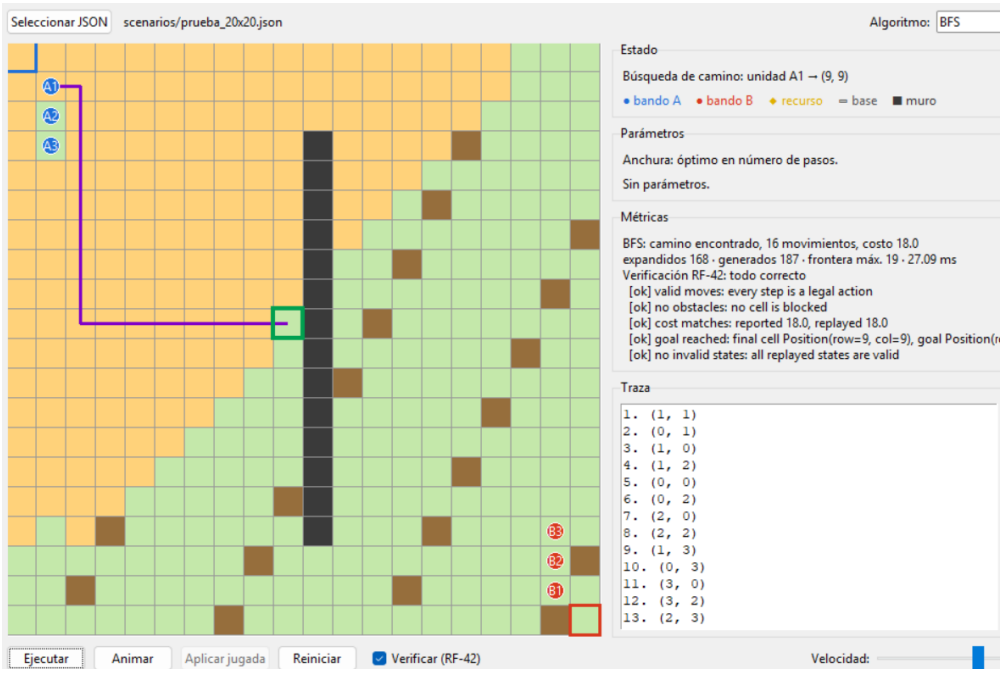


Figura 3.3: *Panel de métricas y verificaciones tras ejecutar un algoritmo.*

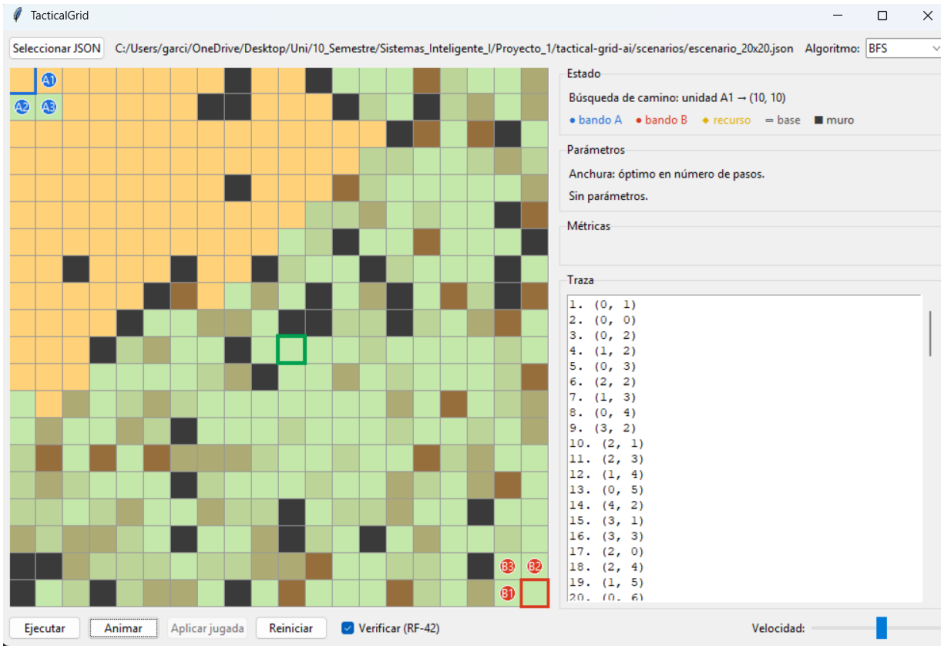


Figura 3.4: *Panel de traza con expansiones, decisiones y acciones de la partida.*

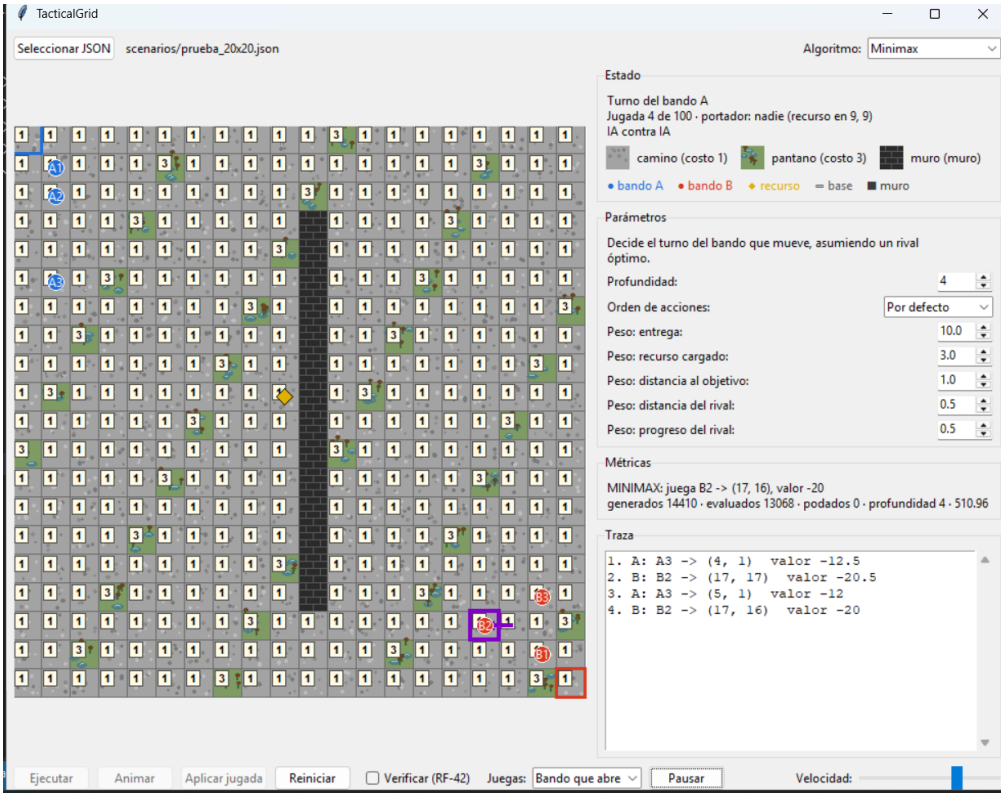


Figura 3.5: *Modo automático de IA contra IA ejecutándose en la interfaz gráfica.*

3.2 El Mapa (Canvas Central)

Cuadro 3.1: *Elementos visuales del mapa*

Elemento	Representación
Terreno transitable	Color interpolado: verde claro (costo mínimo) → marrón (costo máximo); costo escri
Obstáculo (muro)	Gris oscuro `#3a3a3a`, no transitable
Base A / Base B	Borde grueso 3px azul `#1e6fd9` / rojo `#d9381e`; etiqueta *Base A/B*
Recurso	Diamante dorado `#ffc400` con contorno negro
Unidad	Círculo color bando (A=azul `#1e6fd9`, B=rojo `#d9381e`); ID en blanco
Portador	Anillo dorado `#ffc400` grosor 4 alrededor de la unidad
Unidad seleccionada	Anillo negro discontinuo (dash 4,2) alrededor
Meta de búsqueda	Círculo punteado morado `#8000c8` etiquetado *meta*
Camino encontrado	Línea morada `#8000c8` ancho 3 con flecha
Celdas alcanzables	Puntos verdes `#00a050` (al seleccionar unidad en modo mover)
Acción sugerida	Flecha verde `#00a050` ancho 5 (Minimax/α-β) o *esperar*

Colores de terreno: Interpolación lineal entre `CHEAP = (196,232,170)` (verde claro) y `DEAR = (150,110,60)` (marrón) según `(costo - min) / (max - min)`. Leídos **del JSON**, no por nombre de terreno (RNF-03).

3.3 Panel Derecho (Controles)

1. **Estado:** Banner coloreado (azul/rojo) = turno actual; muestra movimiento #/límite, portador del recurso, unidad seleccionada, meta de búsqueda y, en la variante dinámica, unidades en recuperación e inmunidad del portador.
2. **Clic izquierdo sobre el mapa:** Radio buttons:
   - *Seleccionar / mover unidades:* clic sobre una unidad → la selecciona; clic sobre una celda verde → la mueve.
   - *Fijar meta de la búsqueda:* clic derecho (o modo meta + clic izquierdo) fija la meta de búsqueda.
3. **Botones de acción:**
   - *Esperar*: Unidad seleccionada hace `Wait` (costo 0).
   - *Reiniciar partida*: Vuelve al estado inicial del escenario.
   - *Seleccionar JSON*: `filedialog` carga JSON (validación atómica RF-34).
4. **Algoritmo:** ComboBox con búsquedas, decisiones base y decisiones dinámicas:

   BFS, DFS, UCS → sin parámetros extra

   A* → aparece *Heurística* (H1/H2)

   Beam → aparece *Ancho k* (entry numérico)

Minimax, Alfa-beta → aparecen *Profundidad* + *Orden* (Normal/Invertido)

Minimax dinámico, Alfa-beta dinámico → variante con recuperación e inmunidad

Las decisiones también muestran cinco pesos de utilidad. Checkbox *Verificar resultado (RF-42)* ejecuta los checks aplicables tras correr.

5. **Botones de ejecución:**
   - *Ejecutar*: Ejecuta el algoritmo seleccionado con los parámetros actuales.
   - *Aplicar jugada*: Juega la acción devuelta por Minimax/α-β.
   - *Reiniciar*: Regresa al estado inicial del escenario.
6. **Controles de partida:**
   - *Bando del usuario*: elige el bando A o B. Si eliges el bando que no inicia, la IA realiza el primer movimiento.
   - *Partida automática*: la IA juega ambos bandos hasta el final; vuelve a pulsarlo para pausar sin perder el estado.
   - El control de velocidad regula el intervalo entre decisiones.
7. **Traza:**
   - En búsquedas: celdas expandidas en orden.
   - En decisiones: valores de las acciones raíz; un asterisco marca la elegida.
   - En partidas: historial de acciones y pases de turno.

3.4 Interacción: Flujo Típico

1. **Jugar a mano:** seleccionar una unidad → hacer clic en una celda verde → el turno pasa al otro bando.
2. **Ver búsqueda:** modo *Fijar meta* → hacer clic en la meta → elegir el algoritmo → *Ejecutar* → camino dibujado + métricas en pestaña *Resultados*.
3. **Decisión adversarial:** Elegir Minimax/Alfa-beta → *Ejecutar* → flecha verde en mapa + valores en *Resultados* → *Aplicar jugada sugerida*.
4. **Partida automática:** Elegir Minimax/Alfa-beta, o su variante dinámica, ajustar profundidad y velocidad → *Partida automática* → ver IA vs IA.

3.5 Leyenda (Debajo del Mapa)

Muestra dinámicamente:

- Terreno barato (costo mínimo) – verde claro
- Terreno caro (costo máximo) – marrón

- Obstáculo – gris
- Recurso – diamante dorado
- Bandos A/B – círculos azul/rojo
- Portador – círculo gris con anillo dorado

Capítulo 4

Línea de Comandos (CLI)

4.1 Uso Básico

```
uv run python -m tactical_grid <scenario.json> [opciones]
```

Las opciones sobrescriben la sección `prueba` del JSON.

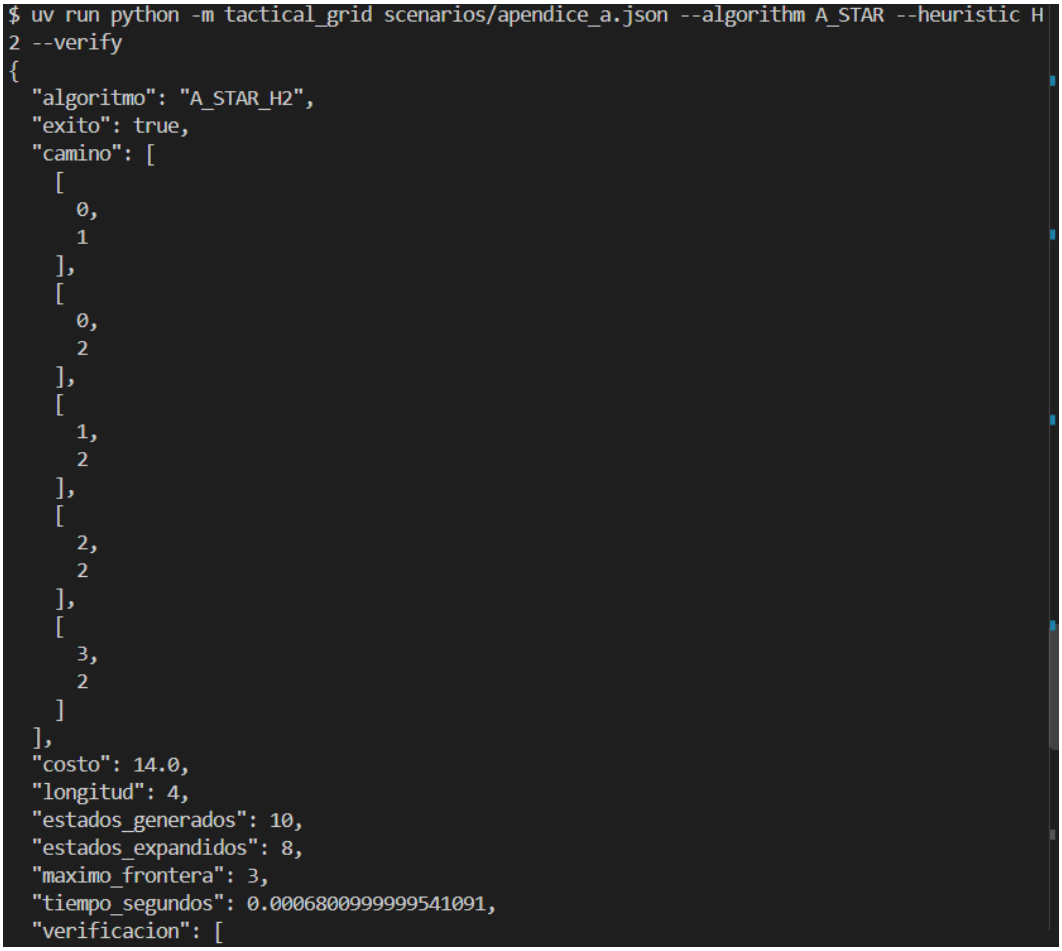


Figura 4.1: *Ejecución de la línea de comandos con métricas y verificación de resultados.*

4.2 Opciones Disponibles

Opción	Tipo	Descripción
`scenario`	positional	Ruta al archivo JSON del escenario
`-mode`	`busqueda`, `adversarial`, `partida`	Modo de ejecución
`-play`	`auto`, `humano`	En `partida`: IA vs IA o persona vs IA
`-side`	`A`, `B`	Bando de la persona con `-play humano`; default A
`-variant`	`base`, `dinamica`	Variante de reglas adversariales; default desde el JSON
`-algorithm`	`BFS`, `DFS`, `UCS`, `A_STAR` (`A*`, `A_ESTRELLA`), `BEAM`, `MINIMAX`, `ALPHA_BETA`	Algoritmo a ejecutar
`-heuristic`	`H1` (Manhattan), `H2` (con obstáculos); valor predeterminado: `H1`	Heurística para A*/Beam
`-k`	int	Ancho de haz (Beam)
`-depth`	int	Profundidad Minimax/α-β (default: `juego.profundidad_maxima_minimax`)
`-order`	`default`, `reversed`	Orden de acciones (Minimax/α-β)
`-unit`	string	Unidad de inicio (búsqueda)
`-goal`	`row,col`	Meta de búsqueda (ej: `3,2`)
`-verify`	indicador	Ejecuta las comprobaciones RF-42 y añade `verificacion` al reporte
`-csv`	`PATH`	Escribe métricas (una fila por corrida)
`-json`	`PATH`	Escribe reportes completos (camino, valores_raiz, checks)

Cuadro 4.1: *Opciones CLI*

4.3 Listas Comma-Separated (Sweep Combinatorio)

Las opciones `-algorithm`, `-heuristic`, `-k`, `-depth`, `-order` aceptan listas separadas por comas y ejecutan **todas las combinaciones** (producto cartesiano) que apliquen.

```
# Beam k=1,2,4,8 con H1
uv run python -m tactical_grid scenarios/caso_05_beam_k.json \
```

```
    --algorithm BEAM --heuristic H1 --k 1,2,4,8 --verify --csv beam.csv

# Minimax frente a alfa-beta, profundidades 1-4, órdenes predeterminado e
↪  invertido
uv run python -m tactical_grid scenarios/caso_07_decision_adversarial.json \
    --algorithm MINIMAX,ALPHA_BETA --depth 1,2,3,4 --order default,reversed
    ↪  --verify

# Sweep completo: todos los algoritmos, heurísticas, k, depths, órdenes
uv run python -m tactical_grid scenarios/apendice_a.json \
    --algorithm BFS,DFS,UCS,A_STAR,BEAM \
    --heuristic H1,H2 --k 1,2,4,8 \
    --depth 3,4,5 --order default,reversed \
    --verify --csv results.csv --json results.json
```

4.4 Variante dinámica

La variante se activa con `-variant dinamica` o mediante `"variante": "dinamica"` en la sección `juego` del JSON. Mantiene el objetivo de llevar el recurso a la base, pero aplica dos reglas adicionales:

- El costo del terreno se convierte en turnos de recuperación propios; una unidad en recuperación no puede actuar hasta quedar libre.
- Un placaje puede robar el recurso y el portador conserva inmunidad durante los turnos indicados por `juego.inmunidad_robo`.

Los algoritmos disponibles para esta variante son Minimax dinámico y Alfa-beta dinámico.

El sweep ejecuta el producto cartesiano de todas las combinaciones válidas (heurística solo para A*/Beam, k solo para Beam, depth/order solo para Minimax/α-β).

```
usage: tactical_grid [-h] [--mode {busqueda,adversarial,partida}] [--algorithm ALGORITHM
                     [--heuristic HEURISTIC] [--k K] [--depth DEPTH] [--order ORDER]
                     [--unit UNIT] [--goal GOAL] [--verify] [--csv PATH] [--json PATH]
                     [scenario]
```

Figura 4.2: *Sweep combinado de algoritmos con exportación de resultados en CSV/JSON.*

4.5 Ejemplos de Uso Común

4.5.1 Búsqueda simple con verificación

```
uv run python -m tactical_grid scenarios/apendice_a.json \
    --algorithm A_STAR --heuristic H2 --verify
```

4.5.2 Adversarial con verificación (check 6)

```
uv run python -m tactical_grid scenarios/caso_07_decision_adversarial.json \
    --algorithm ALPHA_BETA --depth 4 --verify
```

4.5.3 Partida completa desde la terminal

```
# IA contra IA
uv run python -m tactical_grid scenarios/caso_07_decision_adversarial.json \
    --mode partida --play auto --algorithm ALPHA_BETA --depth 3

# Persona contra IA
uv run python -m tactical_grid scenarios/caso_07_decision_adversarial.json \
    --mode partida --play humano --side A --depth 3

# Variante dinámica
uv run python -m tactical_grid scenarios/caso_11_dinamico.json \
    --mode partida --play humano --variant dinamica --depth 3
```

Sin `-play`, `-mode partida` solo describe el estado cargado. Con `-play humano`, las acciones legales aparecen numeradas y se responde con un número; `q` abandona la partida. Con `-play auto`, el reporte incluye el desarrollo completo de la partida.

4.5.4 Sweep de rendimiento (caso 10.5 del informe)

```
uv run python -m tactical_grid scenarios/prueba_15x15.json \
    --algorithm ALPHA_BETA --depth 5,6,7 --csv
    ↪ docs/results/rendimiento_15x15.csv
uv run python -m tactical_grid scenarios/prueba_15x15.json \
    --algorithm MINIMAX --depth 4,5,6 --csv
    ↪ docs/results/rendimiento_15x15_minimax.csv
```

4.5.5 Sweep completo para el informe (todos los casos)

```
=docs/results; ="uv run python -m tactical_grid"
 scenarios/caso_01_bfs_vs_dfs.json --algorithm BFS,DFS,UCS --verify --csv
↪  /caso_01_bfs_vs_dfs.csv --json  /caso_01_bfs_vs_dfs.json
 scenarios/caso_02_bfs_vs_ucs.json --algorithm BFS,DFS,UCS --verify --csv
↪  /caso_02_bfs_vs_ucs.csv --json  /caso_02_bfs_vs_ucs.json
 scenarios/caso_03_obstaculo_heuristica.json --algorithm UCS,A_STAR
↪  --heuristic H1,H2 --verify --csv  /caso_03_obstaculo_heuristica.csv
↪  --json  /caso_03_obstaculo_heuristica.json
 scenarios/caso_04_comparacion_heuristicas.json --algorithm UCS,A_STAR
↪  --heuristic H1,H2 --verify --csv  /caso_04_comparacion_heuristicas.csv
↪  --json  /caso_04_comparacion_heuristicas.json
```

```
$U scenarios/caso_05_beam_k.json --algorithm UCS,BEAM --heuristic H1 --k 1,2,4,8 --verify --csv $R/caso_05_beam_k.csv --json $R/caso_05_beam_k.json
$U scenarios/caso_07_decision_adversarial.json --algorithm MINIMAX,ALPHA_BETA --depth 1,2,3,4,5 --order default,reversed --verify --csv $R/casos_07_10_adversarial.csv --json $R/casos_07_10_adversarial.json
$U scenarios/prueba_20x20.json --algorithm ALPHA_BETA --depth 2,3,4,5,6 --csv $R/rendimiento_20x20.csv
$U scenarios/prueba_15x15.json --algorithm ALPHA_BETA --depth 5,6,7 --csv $R/rendimiento_15x15.csv
$U scenarios/prueba_15x15.json --algorithm MINIMAX --depth 4,5,6 --csv $R/rendimiento_15x15_minimax.csv
$U scenarios/apendice_a.json --algorithm BFS,DFS,UCS,A_STAR,BEAM --heuristic H1,H2 --k 1,2,4,8 --verify --csv $R/apendice_a.csv
```

4.6 Salida JSON (Estructura)

4.6.1 Modo busqueda


```
{
  "algoritmo": "A_STAR_H2",
  "exito": true,
  "camino": [[2,2], [3,2], [4,2], [5,2]],
  "costo": 9,
  "longitud": 3,
  "estados_generados": 84,
  "estados_expandidos": 42,
  "maximo_frontera": 15,
  "tiempo_segundos": 0.00123,
  "verificacion": [
    {"chequeo": "valid moves", "ok": true, "detalle": "every step is a legal action"},
    {"chequeo": "no obstacles", "ok": true, "detalle": "no cell is blocked"},
    {"chequeo": "cost matches", "ok": true, "detalle": "reported 9, replayed 9.0"},
    {"chequeo": "goal reached", "ok": true, "detalle": "final cell [5,2], goal [5,2]"},
    {"chequeo": "no invalid states", "ok": true, "detalle": "all replayed states are valid"}
  ]
}
```


4.6.2 Modo adversarial


```
{
```

```
  "algoritmo": "ALPHA_BETA",
  "profundidad": 4,
  "orden_acciones": "default",
  "accion": "Move(unit_id='A1', dest=Position(row=0, col=0))",
  "valor": 0.0,
  "nodos_generados": 431,
  "nodos_evaluados": 299,
  "nodos_podados": 757,
  "profundidad_alcanzada": 4,
  "valores_raiz": [
    {"accion": "Move(unit_id='A1', dest=Position(row=0, col=0))", "valor": 0.0},
    {"accion": "Move(unit_id='A2', dest=Position(row=0, col=1))", "valor": -2.5}
  ],
  "tiempo_segundos": 0.0045,
  "verificacion": [
    {"chequeo": "Minimax legal actions", "ok": true, "detalle": "all considered actions are legal"}
  ]
}
```


4.6.3 Modo partida


```
{
  "modo": "partida",
  "mapa": [20, 20],
  "turno": "A",
  "portador": null,
  "recurso": [2, 2],
  "unidades": {
    "A1": {"bando": "A", "celda": [0, 1]},
    "A2": {"bando": "A", "celda": [1, 0]},
    "B1": {"bando": "B", "celda": [3, 2]},
    "B2": {"bando": "B", "celda": [2, 3]}
  },
  "bases": {"A": [0, 0], "B": [19, 19]},
  "limite_turnos": 100
}
```

Capítulo 5

Formato de Archivo de Escenario (JSON)

5.1 Estructura Básica

El formato es el del Apéndice A del enunciado. Claves en español (leídas solo en `scenario/`); el código interno usa nombres en inglés.

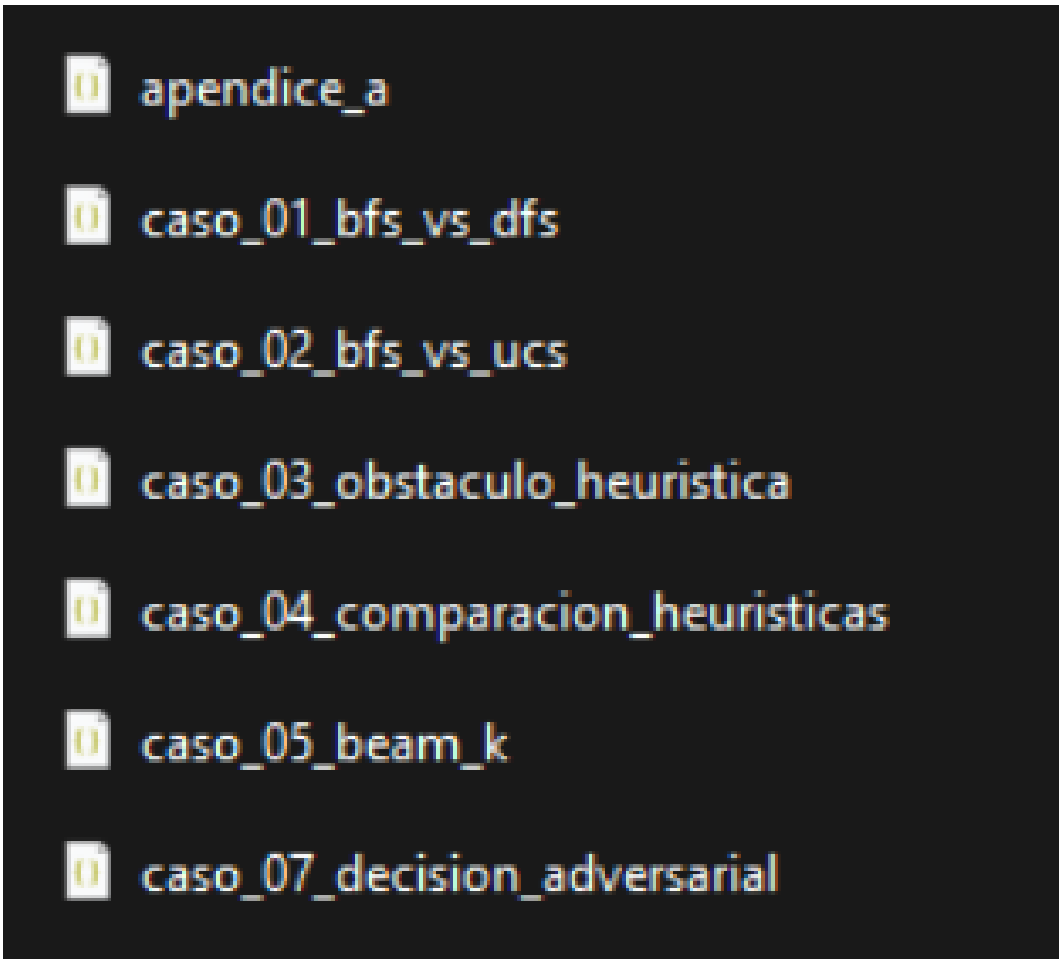


Figura 5.1: *Estructura del escenario JSON cargado y validado por la aplicación.*

```
$ uv run python -m tactical_grid scenarios/apendice_a.json \
    --algorithm A_STAR \
    --heuristic H2 \
    --verify
{
  "algoritmo": "A_STAR_H2",
  "exito": true,
  "camino": [
    [
      0,
      1
    ],
    [
      0,
      2
    ],
    [
      1,
      2
    ],
    [
      2,
      2
    ],
    [
      3,
      2
    ]
  ],
  "costo": 14.0,
  "longitud": 4,
  "estados_generados": 10,
  "estados_expandidos": 8,
  "maximo_frontera": 3,
  "tiempo_segundos": 0.0006674999999631837,
```

Figura 5.2: *Mensaje de validación del escenario mostrando errores o confirmación correcta.*

```
{
  "version": "1.0",
  "mapa": { "filas": 20, "columnas": 20 },
  "tipos_terreno": {
    "camino":   {"costo": 1, "transitable": true},
    "pasto":    {"costo": 2, "transitable": true},
    "bosque":   {"costo": 4, "transitable": true},
    "pantano":  {"costo": 7, "transitable": true},
    "muro":     {"transitable": false}
  },
```

```
  "terreno": [
    ["camino", "camino", "pasto", "pasto"],
    ["camino", "muro",   "bosque", "pasto"],
    ...
  ],
  "bases": { "A": {"fila": 0, "columna": 0}, "B": {"fila": 19, "columna": 19} },
  "recurso": {"fila": 10, "columna": 10},
  "unidades": [
    {"id": "A1", "bando": "A", "tipo": "estandar", "fila": 5, "columna": 5},
    {"id": "B1", "bando": "B", "tipo": "estandar", "fila": 15, "columna": 15}
  ],
  "turno": "A",
  "juego": {
    "portador_recurso": null,
    "profundidad_maxima_minimax": 4,
    "limite_turnos": 100,
    "variante": "base",
    "turnos_recuperacion": 1,
    "inmunidad_robo": 1,
    "utilidad": {
      "entrega": 10,
      "recurso_cargado": 3,
      "distancia_objetivo": 1,
      "distancia_rival": 0.5
    }
  },
  "prueba": {
    "modo": "busqueda",
    "algoritmo": "A_STAR",
    "heuristica": "H2",
    "unidad_inicio": "A1",
    "objetivo": {"fila": 15, "columna": 15},
    "k": 4,
    "orden_acciones": "default"
  }
}
```

5.2 Campos Obligatorios

Campo	Tipo	Descripción
`version`	string	Versión del formato (`"1.0"`)
`mapa.filas`	int > 0	Filas del grid
`mapa.columnas`	int > 0	Columnas del grid
`tipos_terreno`	object	Catálogo: `nombre` $\rightarrow$ `{costo, transitable}`

Continúa

Campo	Tipo	Descripción
`terreno`	string[][]	Matriz `filas` × `columnas`
`bases.A`	`{fila, columna}`	Posición base A
`bases.B`	`{fila, columna}`	Posición base B
`recurso`	`{fila, columna}`	Posición recurso (si nadie lo porta)
`unidades`	array	Lista: `id, bando, tipo, fila, columna`
`turno`	`"A"` | `"B"`	Bando que debe actuar
`juego.portador_recurso`	string | `null`	id unidad que porta, o `null`

Cuadro 5.1: *Campos obligatorios del JSON*

5.3 Campos Opcionales del Equipo (RNF-02)

Cuadro 5.2: *Campos opcionales del equipo (no alteran obligatorios)*

Campo	Default	Descripción
`juego.limite_turnos`	100	Límite de turnos; empate al alcanzar
`juego.profundidad_maxima_minimax`	4	Profundidad para Minimax/α-β
`juego.variante`	`base`	`base` o `dinamica`
`juego.turnos_recuperacion`	1	Turnos propios de recuperación en variante dinámica
`juego.inmunidad_robo`	1	Turnos del rival con inmunidad tras perder el recurso
`utilidad`	10/3/1/0.5	Pesos: `entrega`, `recurso_cargado`, `distancia_objeti`
`prueba.algoritmo`	none	Algoritmo: `BFS`, `DFS`, `UCS`, `A_STAR`, `BEAM`, `MINIMAX`, `ALP`
`prueba.heuristica`	`H1`	`H1` (Manhattan) o `H2` (Obstacle-aware)
`prueba.k`	none	Ancho de haz (Beam)
`prueba.orden_acciones`	`default`	`default` o `reversed`
`prueba.unidad_inicio`	none	id unidad de inicio (búsqueda)
`prueba.objetivo`	none	`{fila, columna}` meta de búsqueda

5.4 Variante dinámica

La variante dinámica se activa con `"variante": "dinamica"` dentro de `juego` o con el argumento CLI `-variant dinamica`. El costo del terreno determina los turnos de recuperación de una unidad y el placaje puede robar el recurso respetando la inmunidad configurada. Si no se especifica la variante, se usa `base`.

5.5 Reglas de Validación (RF-34, RF-35)

1. **Atómica:** Se valida todo antes de construir; cualquier error → `ScenarioError` con la lista completa; no se construye ningún escenario parcial.

2. **9 validaciones mínimas (RF-35) + 1 extra:**

a) Filas/columnas positivas y consistentes con matriz
b) Terrenos usados existen en `tipos_terreno`
c) Transitables tienen costo positivo
d) Bases, recurso, unidades dentro del mapa
e) Ninguna unidad en celda no transitable
f) IDs de unidad únicos
g) Cada unidad en bando `A`/`a` o `B`/`b`
h) Turno es `A`/`a` o `B`/`b`
i) `portador_recurso` es id existente o `null`
j) **Extra:** Ninguna unidad comparte celda con otra

3. **Normalización:** Bandos `A`/`a` → `A`, `B`/`b` → `B`; costos a `float`; terrenos no transitables sin `costo` tolerados.

5.6 Escenarios Incluidos

Cuadro 5.3: *Escenarios en `scenarios/`*

Archivo	Uso
`apendice_a.json`	Ejemplo oficial 4×4
`caso_01_bfs_vs_dfs.json`	Caso 1
`caso_02_bfs_vs_ucs.json`	Caso 2
`caso_03_obstaculo_heuristica.json`	Caso 3
`caso_04_comparacion_heuristicas.json`	Caso 4
`caso_05_beam_k.json`	Caso 5
`caso_07_decision_adversarial.json`	Casos 6–10 (5×7)
`prueba_15x15.json`	Rendimiento 15×15 (3 unidades/bando)
`prueba_20x20.json`	20×20 (3 unidades/bando)
`escenario_03x03.json` .. `escenario_30x30.json`	Escalabilidad

5.7 Crear un Escenario Nuevo

1. Copia `apendice_a.json` a `scenarios/mi_escenario.json`
2. Modifica `mapa.filas`, `mapa.columnas`, `terreno`, `bases`, `recurso`, `unidades`, `turno`
3. Ajusta `tipos_terreno` y `juego` según necesites
4. Ejecuta: `uv run python -m tactical_grid scenarios/mi_escenario.json -verify`
5. Si hay errores, el mensaje lista TODOS los problemas; corrige y repite

No se requiere tocar código: Cualquier JSON válido carga sin modificar el programa (RF-36, RF-37, RNF-03, RNF-04).

Capítulo 6

Reproducir los Experimentos Reportados

Todos los resultados del informe técnico (`docs/results/`) se pueden regenerar ejecutando los comandos exactos que se listan a continuación. Los valores de conteo (generados, expandidos, frontera, costo, longitud) son **determinísticos** e idénticos en cada ejecución; solo `tiempo_segundos` varía (tiempo de ejecución).

6.1 Preparación

```
cd tactical-grid-ai
uv sync                     # instala dependencias
uv run pytest               # verifica que 281 tests pasan
R=docs/results
U="uv run python -m tactical_grid"
mkdir -p $R
```

6.2 Casos 1–2: Búsqueda No Informada y Costos (F2)

```
# Caso 1: BFS vs DFS
$U scenarios/caso_01_bfs_vs_dfs.json --algorithm BFS,DFS,UCS --verify \
  --csv $R/caso_01_bfs_vs_dfs.csv --json $R/caso_01_bfs_vs_dfs.json

# Caso 2: BFS vs UCS (ruta corta vs económica)
$U scenarios/caso_02_bfs_vs_ucs.json --algorithm BFS,DFS,UCS --verify \
  --csv $R/caso_02_bfs_vs_ucs.csv --json $R/caso_02_bfs_vs_ucs.json
```

Valores esperados (derivados a mano, asertados en tests):

Cuadro 6.1: *Caso 1: BFS vs DFS*

Alg	Costo	Movs	Gen	Exp	MaxF	Match
BFS	4	4	39	29	11	Sí
DFS	42	42	152	94	58	Sí
UCS	4	4	39	29	11	Sí

Cuadro 6.2: *Caso 2: BFS vs UCS*

Alg	Costo	Movs	Gen	Exp	MaxF	Match
BFS	25	7	126	104	23	Sí
DFS	153	87	396	287	171	Sí
UCS	17	17	269	239	33	Sí

6.3 Casos 3–5: Búsqueda Informada y Beam (F3)

```
# Caso 3: Obstáculo + heurística
$U scenarios/caso_03_obstaculo_heuristica.json \
  --algorithm UCS,A_STAR --heuristic H1,H2 --verify \
  --csv $R/caso_03_obstaculo_heuristica.csv --json
  ↪ $R/caso_03_obstaculo_heuristica.json

# Caso 4: Comparación heurísticas
$U scenarios/caso_04_comparacion_heuristicas.json \
  --algorithm UCS,A_STAR --heuristic H1,H2 --verify \
  --csv $R/caso_04_comparacion_heuristicas.csv --json
  ↪ $R/caso_04_comparacion_heuristicas.json

# Caso 5: Beam k=1,2,4,8
$U scenarios/caso_05_beam_k.json \
  --algorithm UCS,BEAM --heuristic H1 --k 1,2,4,8 --verify \
  --csv $R/caso_05_beam_k.csv --json $R/caso_05_beam_k.json
```

Valores esperados:

Cuadro 6.3: *Caso 3: UCS, A* h1, A* h2*

Alg	Costo	Movs	Gen	Exp	MaxF	Match
UCS	29	29	307	295	17	Sí
A* h1	29	29	194	166	28	Sí
A* h2	29	29	122	84	38	Sí

Cuadro 6.4: *Caso 4: UCS, A* h1, A* h2*

Alg	Costo	Movs	Gen	Exp	MaxF	Match
UCS	24	22	346	332	20	Sí
A* h1	24	22	230	186	44	Sí
A* h2	24	22	164	115	49	Sí

Cuadro 6.5: *Caso 5: Beam k=1,2,4,8*

k	Éxito	Costo	Movs	Gen	Exp	MaxF	Match
1	No	–	–	9	5	3	Sí
2	Sí	30	16	43	29	6	Sí
4	Sí	28	14	50	41	6	Sí
8	Sí	28	14	46	44	6	Sí

6.4 Casos 6–10: Adversarial (F4)

```
# Caso 6: Estado compuesto (test unitario)
uv run pytest tests/test_case_6.py -v

# Caso 7: Minimax profundidades 1-3
$U scenarios/caso_07_decision_adversarial.json \
  --algorithm MINIMAX --depth 1,2,3 --verify \
  --csv $R/casos_07_10_adversarial.csv --json
  ↪ $R/casos_07_10_adversarial.json

# Caso 8: Minimax frente a alfa-beta, profundidad 4
$U scenarios/caso_07_decision_adversarial.json \
  --algorithm MINIMAX,ALPHA_BETA --depth 4 --verify \
  --csv $R/casos_07_10_adversarial.csv --json
  ↪ $R/casos_07_10_adversarial.json

# Caso 9: Orden de acciones (profundidad 3)
$U scenarios/caso_07_decision_adversarial.json \
  --algorithm ALPHA_BETA --depth 3 --order default,reversed --verify \
  --csv $R/casos_07_10_adversarial.csv --json
  ↪ $R/casos_07_10_adversarial.json

# Caso 10: Profundidades 1-5 (alfa-beta)
$U scenarios/caso_07_decision_adversarial.json \
  --algorithm ALPHA_BETA --depth 1,2,3,4,5 --verify \
  --csv $R/casos_07_10_adversarial.csv --json
  ↪ $R/casos_07_10_adversarial.json
```

Valores esperados (casos 7-10):

Cuadro 6.6: *Caso 7: Minimax profundidades 1-3*

Prof	Acción	Valor	Match
1	A2→(0,1)	3	Sí
2	A2→(0,1)	0	Sí
3	A1→(1,1)	3	Sí

Cuadro 6.7: *Caso 8: Minimax vs Alfa-beta (prof 4)*

Alg	Acción	Valor	Gen	Eval	Podados	Match
Minimax	A1→(0,0)	0	4826	4296	0	Sí
Alpha-beta	A1→(0,0)	0	431	299	757	Sí

Cuadro 6.8: *Caso 9: Orden default vs reversed (prof 3)*

Orden	Acción	Valor	Gen	Eval	Podados	Match
default	A1→(1,1)	3	142	110	106	Sí
reversed	A1→(1,1)	3	191	148	145	Sí

Cuadro 6.9: *Caso 10: Alfa-beta profundidades 1-5*

Prof	Acción	Valor	Eval	Match
1	A2→(0,1)	3	7	Sí
2	A2→(0,1)	0	24	Sí
3	A1→(1,1)	3	110	Sí
4	A1→(0,0)	0	299	Sí
5	A1→(1,1)	3	1583	Sí

(Minimax evals: 7 / 64 / 467 / 4296 / 30994)

6.5 Rendimiento (G13)

```
# 15x15 mapa (3 unidades/bando)
$U scenarios/prueba_15x15.json --algorithm ALPHA_BETA --depth 5,6,7 \
  --csv $R/rendimiento_15x15.csv

$U scenarios/prueba_15x15.json --algorithm MINIMAX --depth 4,5,6 \
  --csv $R/rendimiento_15x15_minimax.csv

# 20x20 mapa
$U scenarios/prueba_20x20.json --algorithm ALPHA_BETA --depth 2,3,4,5,6 \
  --csv $R/rendimiento_20x20.csv
```

Valores esperados (ver la sección “Rendimiento (G13)” del informe técnico).

6.6 Sweep Completo (Todos los Casos 1-10 + Apéndice A)

```
R=docs/results; U="uv run python -m tactical_grid"
$U scenarios/apendice_a.json --algorithm BFS,DFS,UCS,A_STAR,BEAM \
  --heuristic H1,H2 --k 1,2,4,8 --verify \
  --csv $R/apendice_a.csv --json $R/apendice_a.json
```

6.7 Caso 11: Variante dinámica

```
# Comparar Minimax dinámico y Alfa-beta dinámico
$U scenarios/caso_11_dinamico.json \
  --variant dinamica --algorithm MINIMAX,ALPHA_BETA --depth 1,2,3,4 --verify
  ↪ \
  --csv $R/caso_11_dinamico.csv --json $R/caso_11_dinamico.json

# Ejecutar una partida completa con la variante dinámica
$U scenarios/caso_11_dinamico.json \
  --mode partida --play auto --variant dinamica --algorithm ALPHA_BETA
  ↪ --depth 3
```

En la variante dinámica se reportan además los efectos de recuperación e inmunidad del portador. Las métricas de tiempo pueden cambiar entre ejecuciones; las acciones y conteos son deterministas para el mismo escenario y parámetros.

6.8 Verificación de que Todo Pasa

```
# Tests completos
uv run pytest                    # 281 tests, 0 skips
uv run ruff check .              # lint
uv run ruff format --check .     # formato
uv run mypy src                  # tipos strict

# Verificar archivos de resultados generados
ls -la docs/results/
# Debe existir: caso_01_bfs_vs_dfs.csv/json, caso_02_bfs_vs_ucs.csv/json,
↪ ...
# caso_03_obstaculo_heuristica.csv/json,
↪ caso_04_comparacion_heuristicas.csv/json
# caso_05_beam_k.csv/json, casos_07_10_adversarial.csv/json
# caso_11_dinamico.csv/json
# rendimiento_15x15.csv, rendimiento_20x20.csv, apendice_a.csv/json
```

Todos los valores de conteo (generados, expandidos, frontera, costo, longitud) deben coincidir **exactamente** con las tablas de este manual y del informe técnico (`cases.md`). Solo `tiempo_segundos` varía entre ejecuciones.

Capítulo 7

Tests, Lint y Calidad

7.1 Ejecutar Tests

```
# Todos los tests
uv run pytest

# Tests específicos
uv run pytest tests/test_adversarial.py -v
uv run pytest tests/test_cases_f4.py -v
uv run pytest tests/test_case_6.py -v

# Con coverage (opcional)
uv run pytest --cov=src/tactical_grid --cov-report=term-missing
```

Resultado esperado: 281 tests pasan (0 skips).

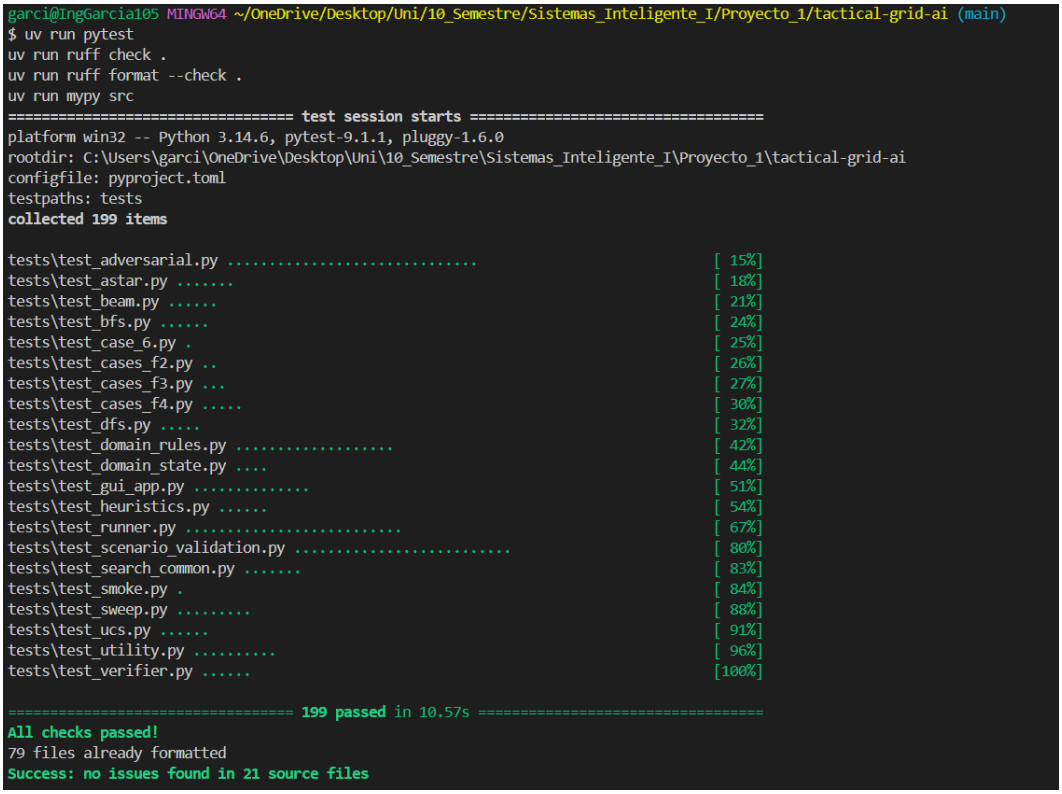


Figura 7.1: *Salida de pytest, ruff y mypy mostrando la validación de calidad del proyecto.*

7.2 Descripción de Suites de Test

Archivo	Nº Tests	Qué prueba
`test_domain_state.py`	5	`GameState`, `Position`, `Board`, inmutabilidad, hash
`test_domain_rules.py`	25	`legal_actions`, `apply`, `successors`, `winner`, `is_terminal`
`test_scenario_validation.py`	20	9 validaciones RF-35 + extra, carga atómica, normalización
`test_bfs.py`	8	Métricas, optimalidad, convenciones BFS
`test_dfs.py`	8	Métricas, orden inverso, convenciones DFS
`test_ucs.py`	10	Métricas, tie-break FIFO, actualización costo, optimalidad
`test_search_common.py`	10	`make_problem`, `reconstruct`, `_normalize`
`test_heuristics.py`	12	Escalado `min_step_cost`, h2≥h1, celdas inalcanzables
`test_astar.py`	15	Optimalidad, equivalencia UCS, reapertura, h1 vs h2
`test_beam.py`	10	Límite k, fallo k=1, determinismo, poda
`test_adversarial.py`	20	Minimax, α-β, métricas RF-30, RF-31, orden
`test_case_6.py`	5	Caso 6: tres estados en la misma posición
`test_cases_f2.py`	8	Casos 1-2 esperado vs obtenido

Continúa

Archivo	Nº Tests	Qué prueba
`test_cases_f3.py`	12	Casos 3-5: heurísticas, Beam
`test_cases_f4.py`	15	Casos 7-10 adversarial
`test_utility.py`	10	Pesos, objetivo dinámico, término rival, terminales
`test_verifier.py`	8	6 checks RF-42 (búsqueda + adversarial)
`test_runner.py`	15	CLI, modos, sweep, export CSV/JSON
`test_sweep.py`	8	Sweep combinatorio, export CSV/JSON
`test_gui_app.py`	8	Ventana, dibujo, clics, auto-mode

7.3 Calidad de Código

```
# Lint
uv run ruff check .

# Formato (check only)
uv run ruff format --check .

# Auto-formatear (modifica archivos)
uv run ruff format .

# Type checking strict
uv run mypy src
```

Estado objetivo: Todo en verde (0 errores, 0 warnings, mypy strict OK).

7.4 Configuración de Ruff (pyproject.toml)

```
[tool.ruff]
line-length = 100
src = ["src", "tests"]

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B"]
```

Reglas activas: E (pycodestyle), F (pyflakes), I (isort), UP (pyupgrade), B (flake8-bugbear).

7.5 Configuración de MyPy (pyproject.toml)

```
[tool.mypy]
strict = true
mypy_path = "src"
```

`strict = true` activa todas las comprobaciones estrictas. El código pasa sin errores.

7.6 Docstrings y Validación Estudiantil

Cada función/método relevante tiene docstring con:

- `Purpose`: Responsabilidad concreta
- `Preconditions`: Condiciones previas
- `Postconditions`: Condiciones posteriores
- `Complexity`: Costo temporal/espacial
- `AI usage`: `Yes/No`
- `AI intervention`: Detalle si hubo IA
- `Student validation`: revisión cruzada realizada por los integrantes del equipo

Ejemplo:

```
def legal_actions(board: Board, state: GameState) -> list[Action]:
    """List the actions available to the side whose turn it is.

    Purpose: enumerate deterministic legal moves (4 directions) and waits per unit.
    Preconditions: `state` belongs to `board` (same units and bounds).
    Postconditions: empty if game over; otherwise, per active unit (sorted by id),
        moves to free passable in-map cells that are not the rival base, then Wait.
    Complexity: O(U * 4 * U) with U units (occupancy lookup is linear in U).
    AI usage: Yes (Claude Code drafted from rules.md).
    AI intervention: none after draft; rules decided by team on 2026-10-14.
    Student validation: revisión cruzada realizada por los integrantes del equipo.
    """
    ...
```

Student validation: Revisión cruzada realizada por los integrantes del equipo.

Capítulo 8

Solución de Problemas Comunes

8.1 Problemas de Instalación y Ejecución

Problema	Causa	Solución
`uv: command not found`	`uv` no instalado o no en PATH	Instalar `uv`: https://docs.astral.sh/uv/getting-started/installation/
`uv sync` falla	Dependencias conflictivas o archivo de bloqueo corrupto	En PowerShell: `Remove-Item uv.lock`; después, ejecutar `uv sync`. En Linux/macOS: `rm uv.lock && uv sync`.
`ModuleNotFoundError: tactical_grid`	Paquete no instalado en entorno	Ejecutar `uv sync` en la raíz del repo
`pytest` no encuentra tests	Directorio incorrecto	Ejecutar desde la raíz del repo (`tactical-grid-ai/`)
Ventana GUI no abre	`tkinter` no disponible	`uv run python -c ïmport tkinter"` para verificar; en Linux: `sudo apt install python3-tk`
Tests de GUI fallan (skip)	Sin display (headless/CI)	Normal: tests de GUI se saltan sin display; no es error

8.2 Errores de Escenario (Exit Code 2)

Mensaje	Causa	Solución
`invalid scenario: validation 1: ...`	JSON inválido: dimensiones, matriz, terrenos	Revisar `mapa.filas/columnas`, `terreno` consistente, nombres en `tipos_terreno`
`validation 2: terrain 'X' needs positive 'costo'`	Terreno transitable sin costo o costo ≤ 0	Añadir `çosto": N` positivo en `tipos_terreno`

Continúa

Mensaje	Causa	Solución
`validation 3: cell (r,c) uses unknown terrain`	Nombre en `terreno` no existe en `tipos_terreno`	Corregir nombre o añadir al catálogo
`validation 4: base A ... outside the map`	Base/recurso/unidad fuera de límites	Corregir coordenadas (0-indexed, < filas/columnas)
`validation 5: unit X on non-passable cell`	Unidad sobre muro/obstáculo	Mover unidad a celda transitable
`validation 6: unit id 'X' is repeated`	IDs duplicados en `unidades`	Usar IDs únicos
`validation 7/8: bando must be 'a' or 'b'`	Bando/turno no es A/a/B/b	Usar `A` o `B` (case-insensitive)
`validation 9: portador_recurso X not existing`	`portador_recurso` no coincide con ningún `id`	Corregir id o poner `null`
`unit X shares cell with unit Y`	Dos unidades en misma celda	Mover una a celda libre

La validación es **atómica**: lista TODOS los errores de una vez; corrige todos y reintenta.

8.3 Problemas de Algoritmos y Resultados

Problema	Causa	Solución
`El algoritmo no encontró camino`	Meta inalcanzable (muros, base rival) o Beam k muy pequeño	Verificar que la meta es alcanzable; aumentar `-k` en Beam; usar UCS/A*
BFS/DFS encuentra camino pero UCS no	Costos no positivos (violación RF-03)	Verificar `tipos_terreno`: transitables deben tener costo > 0
A* con h1 expande mucho / no óptimo	Heurística no admisible o inconsistente	Usar h2 (obstacle-aware); verificar `min_step_cost`

Problema	Causa	Solución
Beam $k = 1$ falla habiendo solución	Beam es incompleto; $k = 1$ poda la única rama buena	Aumentar `-k` (4 u 8); usar A* si se requiere completitud
Minimax/α-β muy lento	Profundidad alta, mapa grande, muchas unidades	Bajar `-depth`; usar α-β en vez de Minimax; mapa $\leq$ 15×15
Resultados no coinciden con esperados	JSON distinto al caso de prueba / semilla distinta	Verificar que se usa el mismo JSON; métricas son determinísticas (solo tiempo varía)
`-verify` falla check 3 (cost matches)	Costo reportado $\neq$ replay	Verificar que `apply` y `cost_at` usan mismo catálogo; sin costos hardcodeados

8.4 Rendimiento y Memoria

- **Minimax puro** > profundidad 5 en 15×15 → > 5 s (supera meta). Usar α-β.
- α-β **profundidad 7** en 15×15 $\approx$ 0.86 s (dentro de meta 5 s).
- **Beam** $k = 1$ puede fallar aunque exista solución (incompleto por diseño).
- **Memoria:** BFS/DFS/UCS/A* usan $O(V)$ espacio (V = estados alcanzables $\leq$ filas $\times$ columnas).
- **Sweeps grandes:** Producto cartesiano de listas → muchas corridas. Ejecutar por lotes o aumentar timeout.

8.5 Obtener Ayuda

1. Revisar `README.md` y este manual.
2. Ejecutar `uv run pytest` para verificar integridad.
3. Revisar `docs/technical-report.md` para detalles de algoritmos y decisiones.
4. Revisar `.context/decisions.md` para decisiones registradas.
5. Si es bug del código: abrir issue en https://github.com/df1903/tactical-grid-ai/issues.
6. Para dudas de la materia: consultar al profesor de Sistemas Inteligentes I.

8.6 Declaración de apoyo de IA

El equipo utilizó Claude Code (Claude Sonnet 5.5) como apoyo para consultas técnicas, redacción, organización y revisión de este manual. La herramienta no fue el desarrollador principal: el equipo definió el alcance, tomó las decisiones, verificó los resultados y revisó el contenido final. Los integrantes del equipo son responsables de explicar y defender toda la información presentada en este documento.

Apéndice A

Referencia Rápida de Comandos

Cuadro A.1: *Resumen de opciones CLI*

Opción	Descripción
`-mode`	`busqueda`, `adversarial`, `partida`
`-algorithm`	`BFS`, `DFS`, `UCS`, `A_STAR`, `BEAM`, `MINIMAX`, `ALPHA_BETA`
`-heuristic`	`H1` (Manhattan), `H2` (Obstacle-aware)
`-k`	Ancho de haz (Beam)
`-depth`	Profundidad Minimax/α-β
`-order`	`default` o `reversed`
`-unit`	Unidad de inicio (búsqueda)
`-goal row,col`	Meta de búsqueda
`-verify`	Ejecutar checks RF-42
`-csv PATH`	Exportar métricas CSV
`-json PATH`	Exportar reportes completos JSON

Apéndice B

Referencia Rápida de JSON

Campo	Obligatorio	Descripción
`version`	Sí	Versión del formato
`mapa.filas`	Sí	Filas del grid
`mapa.columnas`	Sí	Columnas del grid
`tipos_terreno`	Sí	Catálogo: costo, transitable
`terreno`	Sí	Matriz de terrenos
`bases.A/B`	Sí	Posiciones bases
`recurso`	Sí	Posición recurso
`unidades`	Sí	id, bando, tipo, fila, columna
`turno`	Sí	`A` o `B`
`juego.portador_recurso`	No	id unidad o `null`
`juego.limite_turnos`	No	Límite de turnos (default 100)
`juego.profundidad_maxima_minimax`	No	Profundidad para Minimax
`utilidad`	No	Pesos: entrega, recurso_cargado, distancia_objetivo, distancia_rival
`prueba`	No	Configuración de prueba (modo, algoritmo, heurística, k, depth, order, unidad_inicio, objetivo)

Cuadro B.1: *Campos del formato JSON de escenario*

Apéndice C

Referencia Rápida de Algoritmos

Cuadro C.1: *Resumen de algoritmos implementados*

Algoritmo	Tipo	Complejidad	Óptimo
BFS	No informada	$\mathcal{O}(V)$	Solo en movimientos
DFS	No informada	$\mathcal{O}(V)$	No
UCS	Coste uniforme	$\mathcal{O}(V \log V)$	Sí (costo)
A* (h1/h2)	Informada	$\mathcal{O}(V \log V)$	Sí (con h admisible)
Beam (k)	Memoria limitada	$\mathcal{O}(L \cdot k \cdot b \log(kb))$	**No** (incompleto)
Minimax	Adversarial	$\mathcal{O}(B^D)$	Sí (horizonte)
Alpha-beta	Adversarial	$\mathcal{O}(B^{D/2})$ mejor	Sí

Apéndice D

Diagramas de Referencia

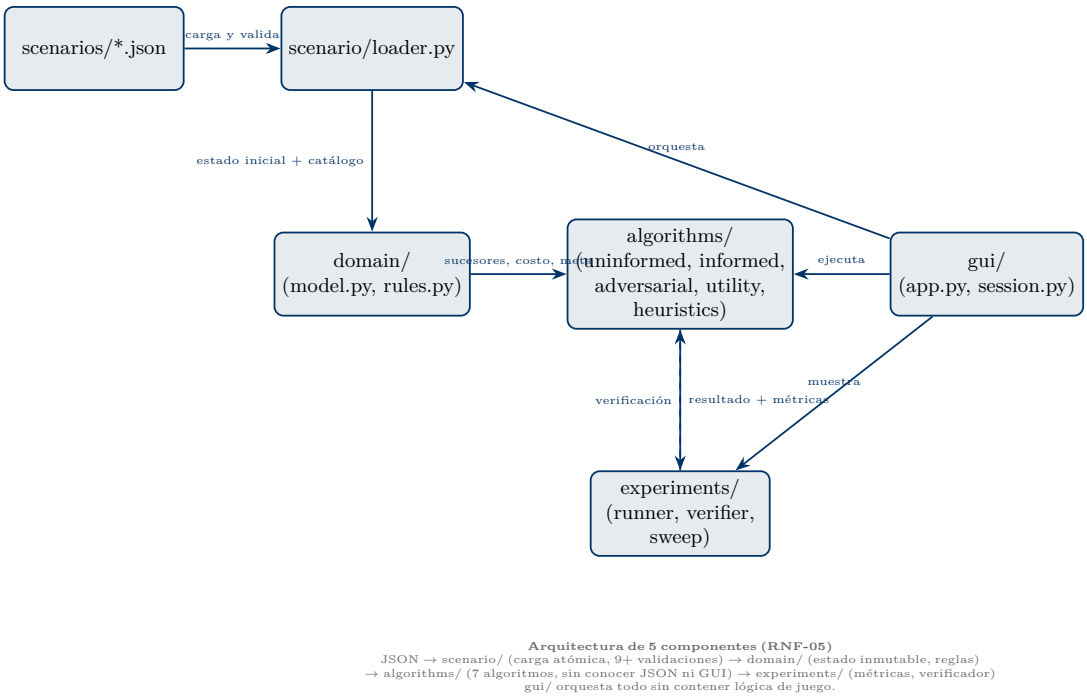


Figura D.1: *Arquitectura general del sistema.*

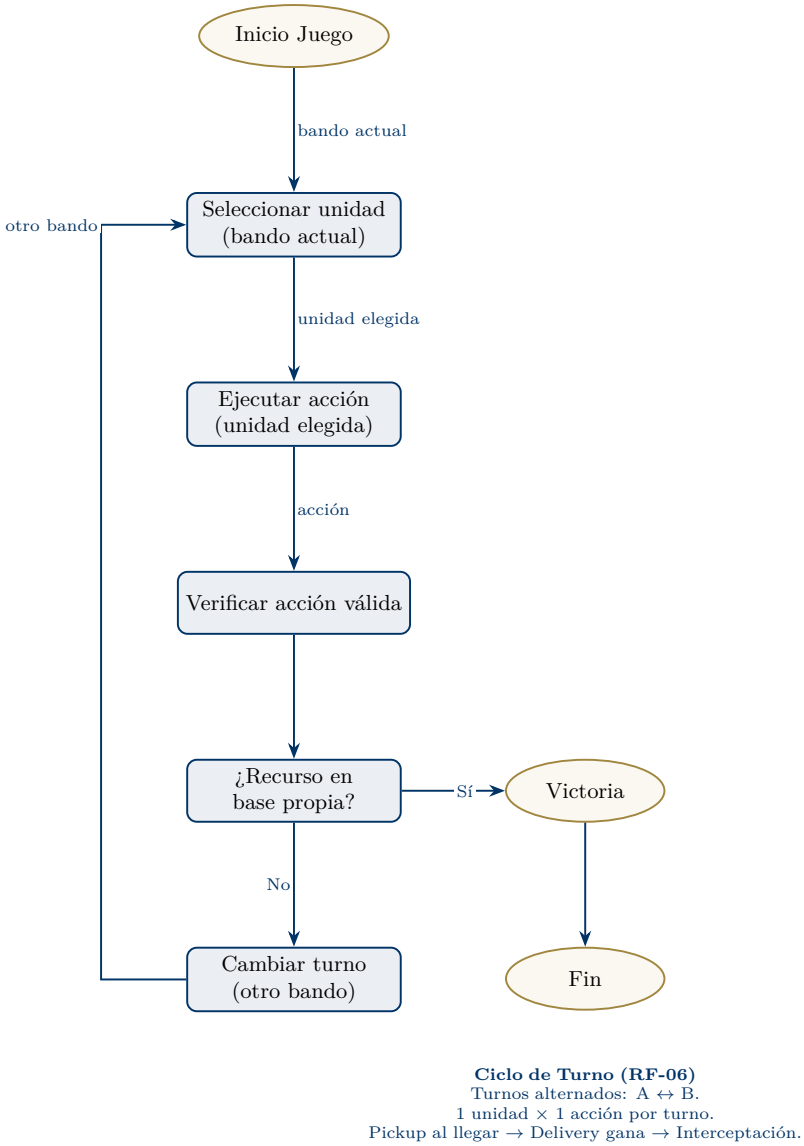


Figura D.2: *Ciclo de turno de la partida.*

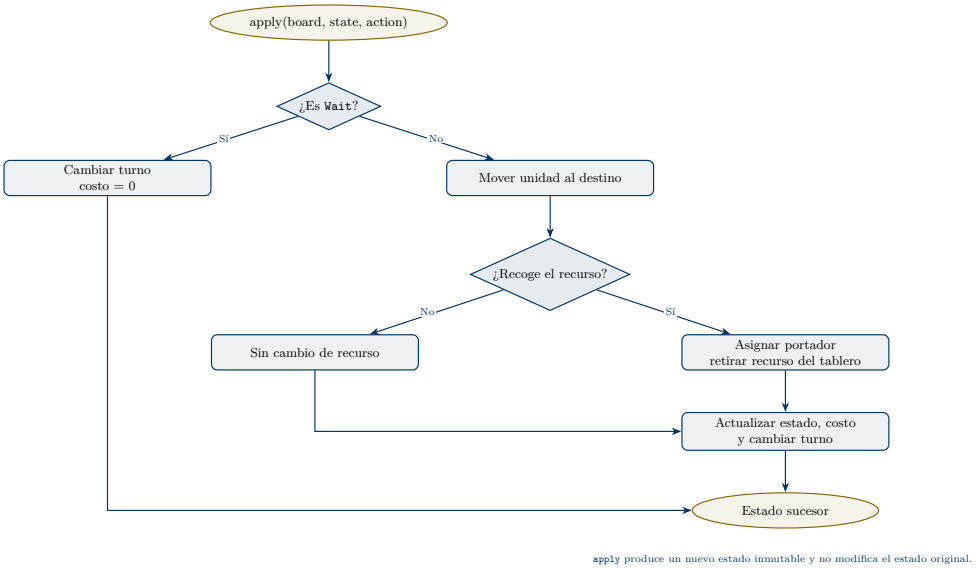


Figura D.3: *Flujo de aplicación de acciones.*